Universidade Federal de Minas Gerais
Departamento de Ciência da Computação
Sistemas Operacionais
Trabalho Prático #2 – Memória Virtual

Introdução

Este trabalho tem por objetivo fazer com que os alunos tenham contato com o tipo de código usado em simuladores e apliquem os conceitos de memória virtual vistos em sala de aula. Deve ser feito em dupla.

Neste trabalho você deve implementar um simulador de memória virtual. Ao contrário do trabalho anterior, este programa não será implementado diretamente no sistema operacional, mas nele você deverá implementar uma réplica das estruturas de um mecanismo de gerência de memória virtual.

Princípio geral

Seu programa deverá ser implementado em C (não serão aceitos recursos de C++, como classes da biblioteca padrão ou de outras fontes). O simulador receberá como entrada um arquivo que conterá a sequência de endereços de memória acessados por um programa real (na verdade, apenas um pedacinho da sequência total de acessos de um programa). Esses endereços estarão escritos como números hexadecimais, seguidos por uma letra R ou W, para indicar se o acesso foi de leitura ou escrita, respectivamente. Ao iniciar o programa, será definido o tamanho da memória (em quadros) para aquele programa e qual o algoritmo de substituição de páginas a ser utilizado. O programa deve, então, processar cada acesso à memória para atualizar os bits de controle de cada quadro, detectar falhas de páginas (*page faults*) e simular o processo de carga e substituição de páginas. Durante todo esse processo, estatísticas devem ser coletadas, para gerar um relatório curto ao final da execução.

Forma de operação

O programa, que deverá se chamar `simulador`, deverá ser iniciado com quatro argumentos:

```
simulador lru arquivo.log 4 128
```

Esses argumentos representam, pela ordem:

1. o algoritmo de substituição a ser usado;
2. o arquivo contendo a sequência de endereços de memória acessados (`arquivo.log`, nesse exemplo);
3. o tamanho de cada página/quadro de memória, em kilobytes – faixa de valores razoáveis: de 2 a 64;
4. o tamanho total da memória física disponível para o processo, também em kilobytes – faixa de valores razoáveis: de 128 a 16384 (16 MB).

Formato da saída

Ao final da simulação, quando a sequência de acessos à memória terminar, o programa deve gerar um pequeno relatório, contendo:

- a configuração utilizada (definida pelos quatro parâmetros);
- o número total de acessos à memória contidos no arquivo;
- o número de *page faults* (páginas lidas);
- o número de páginas "sujas" que tiveram que ser escritas de volta no disco (lembrando-se que páginas sujas que existirem no final da execução não precisam ser escritas).

Um exemplo de saída poderia ser da forma (valores completamente fictícios):

```
prompt> tp2virtual lru arquivo.log 4 128
Executando o simulador...
Arquivo de entrada: arquivo.log
Tamanho da memória: 128 KB
Tamanho das páginas: 4 KB
Técnica de reposição: lru
Total de acessos à memória: 1964
Páginas lidas: 520
Páginas escritas: 352
```

Formato do arquivo de entrada

Como mencionado anteriormente, cada linha contém um endereço de memória acessado, seguido das letras R ou W, indicando um acesso de leitura ou escrita, respectivamente. Por exemplo, as linhas a seguir foram retiradas de um dos arquivos utilizados:

```
0785db58 W
000652d8 R
0005df58 W
000652e0 R
0785db50 W
000652e0 R
31308800 R
00062368 R
```

Os arquivos fornecidos representam o registro de todas as operações de acesso à memória observadas durante a execução real de alguns programas considerados significativos de classes de programas reais:

- `compilador.log`: execução de um compilador, que normalmente utiliza um grande número de estruturas de dados internas complexas;
- `matriz.log`: um programa científico que utiliza cálculos matriciais relativamente simples, mas sobre grandes matrizes e vetores;

- `compressor.log`: um programa de compressão de arquivos, que usa estruturas de dados mais simples;
- `simulador.log`: um simulador de partículas, que executa cálculos complexos sobre estruturas relativamente simples.

Todos os arquivos estão compactados (.zip) e devem ser descompactados antes de serem usados.

A leitura do arquivo pode ser feita com o comando `scanf()`, como no trecho a seguir:

```
unsigned addr;
char rw;
fscanf (file, "%x %c", &addr, &rw);
```

Apesar dos arquivos padrão serem suficientes para realizar uma análise razoável, o grupo é incentivado a gerar seus próprios arquivos de simulação de acesso. Para isso, basta criar um arquivo de texto onde cada linha contenha um endereço hexadecimal de 32 bits seguido por um espaço e uma letra R ou W, representando uma leitura ou escrita, respectivamente. Esses arquivos podem ser gerados manualmente ou com o auxílio de pequenos scripts que simulem padrões específicos de acesso à memória (como acessos sequenciais, aleatórios, com localidade espacial ou temporal, etc.). Explique no relatório como os arquivos adicionais foram produzidos.

Determinação da página a partir do endereço

Como visto em sala, para se identificar a página associada a um endereço, basta descartar os s bits menos significativos do endereço. Se a página fosse fixada em 4 KB, s seria sempre 12. Entretanto, para se implementar um tamanho de página variável, s deve ser calculado a cada execução. Para simplificar, vamos assumir que o tamanho da página será sempre fornecido como uma potência de 2 (não é necessário checar). O código para se determinar s pode ser:

```
unsigned s, page_size, tmp;
/* Derivar o valor de s: */
tmp = page_size;
s = 0;
while (tmp > 1) {
    tmp = tmp >> 1;
    s++;
}
```

Nesse caso, para um endereço `addr`, a página pode ser determinada simplesmente fazendo-se `page = addr » s;`

Seleção de Algoritmos de Substituição de Páginas

Cada grupo deverá selecionar quatro algoritmos de substituição de páginas para implementar e comparar durante o desenvolvimento do trabalho. A escolha deve seguir uma

lógica que permita observar diferentes comportamentos e impactos no desempenho da memória virtual. Os algoritmos devem ser escolhidos com base nos seguintes critérios:

1. **Aleatoriedade (Random):** Escolha ao menos um algoritmo que utilize decisões aleatórias na substituição de páginas, como o algoritmo Random. Esse tipo de política serve como um "ponto de base" para comparação, permitindo entender o impacto de decisões não-determinísticas.

2. **Histórico de Acessos Recentes (por exemplo, LRU):** Selecione um algoritmo baseado na ideia de que páginas usadas recentemente devem permanecer na memória. O algoritmo LRU (Least Recently Used) é o mais clássico dessa categoria. Ele serve para avaliar o benefício de considerar o histórico de uso recente dos quadros.

3. **Frequência de Uso (por exemplo, LFU ou CLOCK):** Inclua um algoritmo que priorize páginas com menos frequência de uso. Isso pode revelar comportamentos diferentes em programas com padrões de acesso muito repetitivos ou espaçados.

4. **Critério Livre:** Escolha um algoritmo que julgar interessante, ou crie seu próprio.

Cada grupo deverá apresentar no relatório:

- Os quatro algoritmos escolhidos, com uma explicação clara da motivação por trás de cada escolha.

- As expectativas de desempenho para cada um, com base nas características dos programas de teste.

- Hipóteses iniciais sobre em quais cenários (por exemplo, memória limitada, páginas grandes, padrões repetitivos) cada algoritmo tende a ter melhor ou pior desempenho.

Implementação da tabela de páginas

Como os endereços são de 32 bits nos arquivos fornecidos, para páginas de 2 KB (as menores que precisam ser consideradas), podemos ter até 21 bits de identificação de página, isto é, uma tabela de página de mais de dois milhões de entradas. Se cada entrada da tabela tiver um inteiro para identificar a página física, teremos um vetor de 8 MB.

Embora não faça diferença do ponto de vista da quantidade de *misses*, a estrutura de tabela de páginas é fundamental em relação ao gasto de memória e desempenho. Assim, vocês devem implementar uma tabela de páginas densa (sem hierarquia), a tabela hierárquica com 2 e três níveis e a tabela invertida. Os resultados devem conter uma avaliação de desempenho das consultas nessas tabelas, assim como o custo de memória para cada configuração.

A estrutura de dados para representar cada quadro físico deve conter campos para registrar atributos como página referenciada, instante do último acesso, página alterada, etc. (os detalhes são parte da implementação e vão depender da forma como vocês implementarem cada algoritmo).

Implementação dos algoritmos de reposição

Os detalhes de outras estruturas de dados que podem vir a ser usadas para os algoritmos são de livre escolha dos implementadores. Vocês devem documentar no relatório como cada algoritmo foi implementado e certificar-se de que o desempenho final do simulador não seja demasiado lento (preferivelmente na ordem de segundos, não dezenas de minutos).

Vocês podem utilizar um contador que inicie em zero e seja incrementado a cada acesso à memória, como a forma de manter o tempo de cada acesso/leitura/escrita, quando necessário.

Para a implementação da política aleatória, as funções `random()` e `srandom()` podem ser usadas para se controlar o gerador de números aleatórios. Para se gerar um número entre 0 e n, a expressão `(random() % n)` é suficiente. Note-se que enquanto houver páginas vazias na memória elas devem ser preenchidas; apenas quando a memória estiver completamente ocupada uma página aleatória deve ser substituída.

Verificação do funcionamento do programa

Apesar da versão a ser entregue precisar gerar apenas o relatório final descrito anteriormente, recomenda-se fortemente que o programa tenha um modo "depuração" onde ele escreva uma linha (ou mais) descrevendo o que é feito a cada acesso à memória. Assim, utilizando-se um arquivo de teste reduzido (e possivelmente escrito especialmente para testar cada caso de operação) vocês podem acompanhar a operação do programa passo-a-passo. Para isso, é permitido prever um quinto parâmetro, opcional, que seja definido quando se deseje que o programa tenha esse comportamento com saída detalhada. A forma desse parâmetro é de escolha dos desenvolvedores. Pode ser uma palavra, como `debug`, pode ser qualquer coisa (apenas para configurar um quinto argumento), ou pode ser um número, que pode ser usado para especificar um grau de detalhamento (números maiores geram mensagens de depuração mais detalhadas, por exemplo).

O que deve ser entregue

Você deve entregar no Moodle um arquivo `.zip` ou `.tgz` contendo o(s) arquivo(s) do código fonte do programa (`.c` e `.h`), um `Makefile` e um relatório sobre o seu trabalho. Inclua os arquivos de teste no processo de entrega!

O relatório deve conter:

- **Um resumo do projeto:** alguns parágrafos que descrevam a estrutura geral do seu código e todas as estruturas importantes.
- **Motivação e escolha dos algoritmos:** justifique brevemente a escolha dos algoritmos de substituição e tipos de tabelas de páginas adotados.
- **Decisões de projeto:** descreva como você lidou com quaisquer ambiguidades na especificação.
- **Uma análise do desempenho dos algoritmos de substituição de páginas e estrutura de dados da tabela de páginas para os vários programas utilizados.** Essa análise de desempenho é a parte mais importante do trabalho e será

responsável por uma fração significativa da nota. Em diversos momentos precisamos comparar algoritmos, determinar o que esperar em diferentes condições. Seu relatório deve avaliar o comportamento dos três algoritmos de reposição de página para os quatro programas que foram disponibilizados, em duas situações:

  - Quando o tamanho da memória cresce, com páginas de 4 KB
  - Quando o tamanho da memória fica constante, mas o tamanho das páginas varia de 2 KB a 64 KB (em potências de 2).
  - **IMPORTANTE:** utilize diferentes recursos como gráficos e tabelas para mostrar as diferenças nas execuções e explicar as suas conclusões. Separe também uma seção para explicar o programa que você criou e qual foi a lógica por trás dele.

- **Conclusão:** os resultados foram os esperados? Houve alguma divergência? Quais foram as causas observadas para essas divergências? O que poderia ser feito de forma diferente ou melhorada em trabalhos futuros?

Finalmente, embora você possa desenvolver o seu código em qualquer sistema que quiser, certifique-se que ele execute corretamente na máquina virtual com o sistema operacional Ubuntu Linux.

Considerações Finais

Mais vale um trabalho curto e bem feito do que um longo, tedioso e sem resultados interessantes. Priorize a clareza, a objetividade e a relevância do conteúdo apresentado. Inclua apenas gráficos que realmente contribuam para a análise; evite repetições ou variações pouco informativas. Lembre-se de que o propósito deste trabalho não é apenas avaliar a sua habilidade de programação, mas principalmente sua capacidade de sintetizar dados brutos, realizar análises coerentes e apresentá-las de forma clara e interessante.

Demonstre no relatório que você compreendeu os impactos das decisões de projeto, soube avaliar as diferenças de desempenho entre algoritmos e estruturas de dados, e conseguiu tirar conclusões relevantes a partir dos experimentos realizados.

- **Dúvidas:** usem o Moodle (minha.ufmg).
- Comece a fazer o trabalho logo, pois apesar do programa final ser relativamente pequeno, o tempo não é muito e o prazo de entrega não vai ficar maior do que ele é hoje (independente de que dia é hoje).
- Vão valer pontos clareza, qualidade do código e, obviamente, a da documentação.
- **Somente um trabalho por dupla deve ser entregue.**

Distribuição de pontos do trabalho

Elemento	**O que é esperado?**	**Nota Final**
Código	Código limpo, bem organizado, com comentários somente nas partes relevantes, compilando e passando em todos os testes.	20%
Documentação - Resumo	Descrição da organização do código, como estruturas auxiliares criadas, organização de funções ou qualquer detalhe que julgar importante.	10%
Documentação - Motivação	Ao menos um parágrafo por algoritmo implementado, explicando o funcionamento e o motivo da escolha.	10%
Documentação - Decisões de Projeto	Pode não ser necessário preencher esta seção, mas se para seu projeto funcionar algum tipo de manipulação esperta precisou ser feita, uma boa documentação aqui pode ser a diferença entre zerar ou não o trabalho.	0%
Documentação - Análise de Desempenho	Texto sucinto e claro explicando as ocasiões em que determinado algoritmo é melhor e o porquê, além de gráficos exemplificando a explicação.	50%
Documentação - Conclusão	Compilação de todas as informações relevantes do trabalho de forma sucinta, mas completa.	10%